Consensus Algorithms

What Are Consensus Algorithms?

Consensus algorithms are **protocols used in distributed systems** to ensure that multiple nodes agree on a single value or state, even in the presence of failures. Without consensus, nodes may **disagree**, causing inconsistency.

Definitions

Agreement

- All non-faulty nodes decide on the **same value**.

Validity

- The chosen value must be **proposed by some node**.

Termination

- All non-faulty nodes **eventually decide**.

Deterministic Algorithms

- Always produce the **same output** and follow the **same execution path** for the same inputs.
- **No randomness**; behavior is fully predictable.
- *Example:* Node always picks the first value in a queue.

Asynchronous Networks

- Messages can be **delayed arbitrarily**; nodes take **arbitrary time** to process.
- No global clock or timing guarantees; messages may arrive **out of order**.
- Nodes **cannot tell** a slow node from a crashed one.
- *Example:* Internet (variable latency) vs LAN (predictable → synchronous).

Limits of Deterministic Consensus in Unreliable Networks

Two Generals Problem (2 nodes, unreliable messages)

- Imagine **two generals**, A and B, who must attack at the same time.
- They send messengers back and forth to agree on a time.
- **Problem:** Any message can be **lost**.
  - A sends: “Attack at 6 pm.”
  - B receives and confirms: “Got it, attack at 6 pm.”
  - But if that confirmation is lost, A **doesn’t know that B knows**.
- **Infinite loop:** You can keep sending confirmations forever, but because messages may be lost, you can **never reach 100% certainty**.

Key idea: Absolute agreement is **impossible** if messages can be lost.

FLP Impossibility (≥3 nodes, asynchronous network)

- Now imagine **3 or more nodes** trying to agree on a value (e.g., yes/no).
- Network is **asynchronous**, meaning messages can take **arbitrarily long** to arrive.
- Suppose **node C crashes** or is just very slow.
- Then:
  1. If nodes **wait for C**, they may **never terminate**, because C might be extremely slow.
  2. If nodes **ignore C**, they risk **violating agreement**, because C might have a different value and could have influenced the consensus.

Key idea: Even if only **one node may crash**, in a purely asynchronous network, no deterministic algorithm can **guarantee both termination and agreement**.

Workarounds

Partial synchrony

- The network may behave badly at first, but eventually:
  - Messages arrive within some maximum time
  - Nodes respond reasonably quickly
- This **“eventual predictability”** is called **partial synchrony**.
- Once the network is partially synchronous, timeouts become meaningful, and consensus protocols like **Raft or Paxos** can terminate.

Timeouts

- A **timeout** says: “If I don’t hear from someone in X ms, I assume they failed.”
- This is a **guess**, not certainty.
- FLP assumes you cannot make any timing assumptions. So **using a timeout breaks the FLP assumption**.

💡 Real networks aren’t truly asynchronous — usually messages arrive eventually. So timeouts **work in practice**, even if they don’t guarantee correctness in theory.

Leader election

- To simplify consensus: choose **one leader** (coordinator).
- Leader handles:
  - Serializing updates
  - Sending heartbeats
  - Coordinating replication
- Leader election uses **timeouts**, which again breaks FLP assumptions.
- Once the leader is stable, the system can make progress efficiently.

Bully Algorithm

Based on the concept of **"brute force"** tied to a unique Node ID. The node with the highest ID always wins.

- **Mechanism:**
  - When a node detects the leader is offline, it sends an "Election" message to all nodes with a **higher ID** than its own.
  - If it receives a response from a higher-ID node, it stands down.
  - If no response is received, it proclaims itself the leader and sends a "Victory" message to all other nodes.
- **Strengths:** Very fast if high-ID nodes are stable and reliable.
- **Weaknesses:** Generates high network traffic ($O(n^2)$ messages) and suffers if the highest-ID node "flaps" (constantly crashes and restarts).

Ring Algorithm

Based on a **logical circle structure**. Each node is aware only of its immediate successor.

- **Mechanism:**
  - When a node detects a leader failure, it creates an "Election" message containing its own ID and sends it to its neighbor.
  - Each node receiving the message adds its ID to the list and passes it forward.
  - Once the message returns to the initiator (completing the full circle), the node identifies the highest ID in the list as the winner.
  - A second round of messages is sent to inform the entire ring of the new leader.
- **Strengths:** More organized and predictable; prevents network "flooding."
- **Weaknesses:** Slower performance (requires a full cycle) and vulnerable if another node in the ring fails during the election process.

Raft

https://thesecretlivesofdata.com/raft/

References

- https://thesecretlivesofdata.com/raft/
- https://raft.github.io/